DBT TRAINING

Beginner · Module 04 · 90 min

Materializations

The most consequential decision in any dbt project. Drives cost, performance, freshness, and pipeline reliability.

The Four Materializations

Define the result object of the model.sql file

Materialization	What dbt creates	Full rebuild?	Use case
`view`	`CREATE OR REPLACE VIEW`	✅ view definition only	Staging, lightweight transforms
`table`	`DROP + CREATE TABLE AS SELECT`	❗ always rebuild	Small Silver models SCD1, Gold marts, reference tables
`incremental`	`MERGE INTO` or `INSERT`	New/changed rows	Large Silver facts, append-heavy
`ephemeral`	Nothing (becomes a CTE)	N/A	Intermediate steps, no table needed

1. For SCD2 type models dbt provides **snapshots**. We will cover this later.
2. Force **incremental** models to rebuild with **dbt run --full-refresh**

`view` and `table` — The Simple Two

view

```
-- dbt compiles to:
CREATE OR REPLACE VIEW
  SILVER_DEV.TESTING__dev_jane.stg_hubspot__contacts
AS
SELECT contact_id, email, created_at
FROM BRONZE.HUBSPOT.contacts
```

✓ Always reflects latest source data

✓ Zero storage cost

✗ Recomputes on every query — slow for complex transforms

By convention: All staging models. Views are cheap wrappers — rename, cast, nothing more.

table

```
-- dbt compiles to:
DROP TABLE IF EXISTS SILVER.PUBLIC.dim_pipeline;
CREATE TABLE SILVER.PUBLIC.dim_pipeline AS
SELECT ...
```

✓ Fast to query — no recomputation

✗ Full rebuild on every run — expensive for large tables

By convention: Gold marts (small aggregates) and Silver dimensions (medium size, nightly rebuild acceptable).

`incremental` — The Critical One

```
{{ config(
    materialized     = 'incremental',
    unique_key       = 'contact_key',
    on_schema_change = 'sync_all_columns'    -- our standard
) }}

SELECT contact_key, hubspot_contact_id, email, updated_at
FROM {{ ref('stg_hubspot__contacts') }}

{% if is_incremental() %}
    WHERE updated_at > (SELECT MAX(updated_at) FROM {{ this }})
{% endif %}
```

First run

`is_incremental()` = False → WHERE skipped → full load → table created

Subsequent runs

`is_incremental()` = True → WHERE applies → MERGE only new/changed rows

Incremental: First Run vs Subsequent Runs

The Compiled MERGE Statement

What Snowflake actually receives for an incremental model:

```
MERGE INTO SILVER.PUBLIC.dim_contact AS DBT_INTERNAL_DEST
USING (
    SELECT contact_key, hubspot_contact_id, email, updated_at
    FROM SILVER_DEV.TESTING__dev_jane.stg_hubspot__contacts
    WHERE updated_at > (SELECT MAX(updated_at) FROM SILVER.PUBLIC.dim_contact)
) AS DBT_INTERNAL_SOURCE
ON DBT_INTERNAL_DEST.contact_key = DBT_INTERNAL_SOURCE.contact_key

WHEN MATCHED THEN UPDATE SET
    hubspot_contact_id = DBT_INTERNAL_SOURCE.hubspot_contact_id,
    email              = DBT_INTERNAL_SOURCE.email,
    updated_at         = DBT_INTERNAL_SOURCE.updated_at

WHEN NOT MATCHED THEN INSERT (contact_key, hubspot_contact_id, email, updated_at)
VALUES (DBT_INTERNAL_SOURCE.contact_key, ...)
```

Find this in `target/compiled/analytics/models/silver/dim_contact.sql`

`on_schema_change` Options

Value	Behaviour	Recommendation
`ignore` *(default)*	New columns in SELECT are silently dropped	❌ Never — silent data bug
`fail`	Errors if SELECT has new columns	For strict environments
`sync_all_columns`	Adds new columns, removes deleted ones	⚠️ Careful, old data is gone
`append_new_columns`	Adds new columns only, never removes	Gives flexibility. Nothing is lost

Use append_new_columns for int_models

Gives flexibility and reduces need to change models at scale if API changes. Focus on silver models to apply new business rules and apply contracts for stability there.

`ephemeral` — The CTE equivalent

```sql
{{ config(materialized='ephemeral') }}

SELECT
    contact_id,
    LOWER(email) AS email_clean
FROM {{ source('hubspot', 'contacts') }}
```

What happens

No object is created in Snowflake. When another model references this via `{{ ref() }}`, dbt inlines it as a CTE.

When NOT to use

If multiple models reference the same ephemeral model, the computation is repeated in each one as a separate CTE. Use a view instead.

In practice: Ephemeral is rarely used. Prefer views for intermediate staging — they're queryable for debugging. An ephemeral model cannot be directly queried.

Snowflake-specific: Dynamic Tables

Snowflake's native alternative to incremental models — the database engine manages refresh automatically.

```
models:
  - name: fct_daily_revenue
    config:
      materialized: dynamic_table
```

Write a plain `SELECT` — no `{% if is_incremental() %}` needed. Snowflake handles the incremental logic.

Advantage

Less code complexity. No `unique_key` or merge strategy to configure.

Trade-off

Less control over refresh timing and logic. No `--full-refresh` override.

Not used in this project today. Standard is `incremental` with `merge` strategy. Mention only if asked — it's a sign Snowflake is absorbing some of what dbt does manually.

Where to Configure Materializations

`dbt_project.yml` lowest priority → `schema.yml` overrides project → `{{ config() }}` highest priority · last wins

`dbt_project.yml` default

Project-wide defaults. Applies to all models. Lives in project root.

```yaml
# dbt_project.yml
models:
  analytics:
    +materialized: view    # all
    silver:
      +materialized: table
      facts:
        +materialized: incrementa
```

`schema.yml` recommended

Per-model config alongside tests, column descriptions & grants. In model folders.

```yaml
# models/silver/schema.yml
models:
  - name: dim_contact
    config:
      materialized: incremental
      unique_key: contact_key
      on_schema_change: sync_all_
    columns:
      - name: contact_key
        description: "Surrogate k
        tests: [not_null, unique]
```

`model.sql` highest priority

Inline at the top of the model. Overrides both levels above. For development only.

```sql
{{ config(
    materialized     = 'increment
    unique_key       = 'contact_k
    on_schema_change = 'sync_all_
) }}

SELECT contact_key, email, update
FROM {{ ref('stg_hubspot__contact
```

Why schema.yml? Tests, column descriptions, grants, and materialization config all live in one file. Splitting config between model SQL and YAML creates drift. Move `{{ config() }}` to `schema.yml` before merging — the SQL file should be pure logic.

Exercise — Code Review (25 min)

Scenario: A colleague pushed 3 new models. `dbt build` is now broken and they've asked for your help. Run the project, read the code carefully, and find all the problems. There are **5 bugs** across the 3 models. For each bug: explain what's wrong, why it matters, and write the fix.

Model 1 — int_pipeline_stages.sql

```
{{ config(materialized='table') }}
SELECT pipeline_stage_id, stage_name, is_closed
FROM {{ source('hubspot', 'pipeline_stages') }}
```

Model 2 — fct_daily_ticket_volume.sql · 50M rows/day

```
{{ config(materialized='table') }}
SELECT ticket_date, COUNT(*) AS ticket_count
FROM {{ ref('dim_ticket') }} GROUP BY 1
```

Model 3 — dim_contact.sql · incremental

Exercise — Solutions

Model 1 · 1 bug

❌ staging as table

```
{{ config(materialized='view') }}

SELECT pipeline_stage_id,
       stage_name,
       is_closed
FROM {{ source('hubspot',
               'pipeline_stages')
```

Staging = view. No storage, always fresh, CI-enforced.

Model 2 · 1 bug

❌ large fact as table

```
{{ config(
    materialized='incremental',
    unique_key='ticket_date',
    on_schema_change='sync_all_co
) }}
SELECT ticket_date,
       COUNT(*) AS ticket_count
FROM {{ ref('dim_ticket') }}
{% if is_incremental() %}
WHERE ticket_date >
  (SELECT MAX(ticket_date)
   FROM {{ this }})
{% endif %}
GROUP BY 1
```

50M rows/day must be incremental — full rebuild is not viable.

Model 3 · 3 bugs

❌ ignore · ❌ · ❌

```
{{ config(
    materialized='incremental',
    unique_key='contact_key',
    on_schema_change='sync_all_co
) }}
SELECT contact_key,
       email,
       updated_at
FROM {{ ref('stg_hubspot__contact
{% if is_incremental() %}
WHERE updated_at >
  (SELECT MAX(updated_at)
   FROM {{ this }})
{% endif %}
```

3 fixes: sync_all_columns · {% if %} · {% endif %}

MODULE 04 COMPLETE

Next: Module 05

Sources and the Medallion Architecture

Prep Q1: What SQL does a table materialization generate?

Prep Q2: What does is_incremental() return on first run?

Prep Q3: Mandatory on_schema_change setting for incremental models?

Prep Q4: Why never use table for a staging model?